Университет ИТМО

Факультет Программной Инженерии и Компьютерной техники

Системы искусственного интеллекта

Лабораторная работа №1

Linear Regression и Logistic Regression

Выполнил:
Ткачев Денис Владимирович
Группа: P3311
ИСУ: 467727
Преподаватель:
Королёва Ю. А.

Санкт-Петербург
2026г.

Содержание

1. Цель работы 3
2. Задание 3
3. Общий подход 3
4. Часть A. Linear Regression 4
   4.1. Данные 4
   4.2. Preprocessing 5
   4.3. Модель 5
   4.4. Результаты 6
5. Часть B. Logistic Regression 7
   5.1. Данные 7
   5.2. Preprocessing 7
   5.3. Модель 8
   5.4. Результаты 10
   5.5. Анализ трат 10
6. Вывод 11

1. Цель работы

Реализовать линейную и логистическую регрессию с обучением градиентным спуском, обучить их на двух датасетах и сравнить с простым baseline.

2. Задание

	Часть A	**Часть B**
Модель	Linear Regression	Logistic Regression
Датасет	Bike Sharing Dataset, файл `day.csv`	Spaceship Titanic, файл `train.csv`
Целевая	`cnt` - число арендованных велосипедов за день	`Transported` - был ли пассажир перенесён в другое измерение
Методы	`__init__`, `fit`, `predict`	`__init__`, `fit`, `predict_proba`, `predict`
Метрики	MAE, RMSE, R^2	Accuracy, Precision, Recall, F1, LogLoss, confusion matrix
Baseline	простой	majority

Ограничения: модели, split, scaler и imputer реализуются самостоятельно. Из sklearn разрешено использовать только `sklearn.metrics`. В части A из признаков нужно удалить `casual` и `registered`, в части B competition `test.csv` не используется.

3. Общий подход

Обе модели написаны на PyTorch без `torch.nn` и `torch.optim`. Модель хранит веса `w` и свободный член `b`, на каждой эпохе считает loss по всей обучающей выборке. Градиент считает `autograd` в `loss.backward()`, веса я обновляю вручную по правилу $w \leftarrow w - \eta \nabla_w L$, где η - скорость обучения `lr`. Начальные веса нулевые, поэтому обучение детерминировано.

В `fit` можно передать тестовую выборку. На ней после каждой эпохи считается loss для графика, веса по ней не обновляются.

Разбиение train/test 70/30 выполняет своя функция. Индексы перемешиваются `torch.randperm` с генератором, у которого фиксирован `seed=42`, поэтому разбиение воспроизводимо.

```
def train_test_split(X, y, test_size=0.3, seed=RANDOM_STATE):
    generator = torch.Generator().manual_seed(seed) # генератор случайных для выбора случайным образом индексов объектов датасета
    idx = torch.randperm(X.shape[0], generator=generator) # случайная перестановка индексов объектов [0, n-1]
    n_test = int(X.shape[0] * test_size)
    test_idx, train_idx = idx[:n_test], idx[n_test:]
    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]

X_train, X_test, y_train, y_test = train_test_split(X_t, y_t)
```

Все статистики preprocessing - медиана, среднее, стандартное отклонение - считаются только по train и применяются к train и test. Если считать их по всей выборке, в обучение попадёт информация о test.

4. Часть A. Linear Regression

4.1. Данные

Датасет содержит `731` строку и `16` столбцов, одна строка - один день 2011-2012 годов. Пропусков и дубликатов нет. Равенство `cnt = casual + registered` выполняется во всех строках.

Среднее	**Медиана**	**Std**	**Min**	**Max**	**Квартили 25% / 75%**
`4504.3`	`4548`	`1937.2`	`22`	`8714`	`3152` / `5956`

Среднее и медиана близки, распределение `cnt` примерно симметрично, поэтому целевую переменную я не преобразовываю. По дням `cnt` растёт от 2011 к 2012 году и колеблется по сезонам.

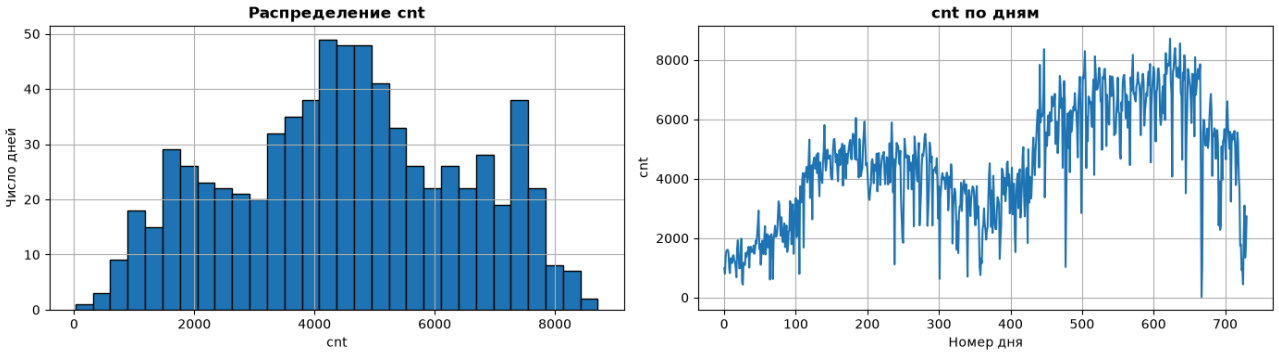


Рис. 1. Распределение `cnt` и `cnt` по дням

4.2. Preprocessing

Признаки	Решение
`casual`, `registered`	удалены, потому что `cnt = casual + registered` и модель получила бы ответ
`instant`	удалён, повторяет номер строки
`dteday`	удалён, год и месяц уже есть в `yr` и `mnth`
`season`, `mnth`, `weekday`, `weathersit`	one-hot кодирование, первая категория отбрасывается. Коды категорий не задают порядка эффекта
`yr`, `holiday`, `workingday`	без изменений, бинарные признаки уже совпадают с one-hot
`temp`, `atemp`, `hum`, `windspeed`	стандартизация средним и стандартным отклонением по train
`cnt`	без изменений, чтобы MAE и RMSE были в числе велосипедов

После кодирования получилось `29` признаков, train - `512` строк, test - `219`.

```
REAL_COLS = ['temp', 'atemp', 'hum', 'windspeed']
num_idx = [feature_names.index(c) for c in REAL_COLS] # номера столбцов REAL_COLS, в тезноре нет имён у столбцов

train_mean = X_train[:, num_idx].mean(dim=0)
train_std = X_train[:, num_idx].std(dim=0)

X_train_s = X_train.clone()
X_test_s = X_test.clone()
X_train_s[:, num_idx] = (X_train[:, num_idx] - train_mean) / train_std
X_test_s[:, num_idx] = (X_test[:, num_idx] - train_mean) / train_std
```

4.3. Модель

Модель предсказывает $\hat{y} = Xw + b$, функция потерь - MSE: $L = \frac{1}{n}\sum_i (\hat{y}_i - y_i)^2$.

```
class LinearRegression:
    def __init__(self, lr=0.01, n_epochs=1000):
        self.lr = lr
        self.n_epochs = n_epochs
        self.w = None
        self.b = None
        self.loss_history = []

    def fit(self, X, y, X_val=None, y_val=None):
        self.w = torch.zeros(X.shape[1], dtype=torch.float32, requires_grad=True) # requires_grad включает подсчёт автоградиента
        self.b = torch.zeros(1, dtype=torch.float32, requires_grad=True)
        self.loss_history = []
        self.val_loss_history = [] # для вывода 2-го графика

        for _ in range(self.n_epochs):
```

```
            y_pred = X @ self.w + self.b
            loss = ((y_pred - y) ** 2).mean()
            loss.backward() # считает градиенты в тензоре через PyTorch Autograd
            # через метод обратного распространения ошибки (backpropagation)

            with torch.no_grad(): # само обновление весов не должно попадать в граф вычислений,
иначе autograd начнёт дифференцировать шаг градиентного спуска.
                self.w -= self.lr * self.w.grad # обновление весов
                self.b -= self.lr * self.b.grad # обновление свободного коэффициента

            # backward() накапливает градиенты при каждом запуске, то есть аккумулирует
градиенты, если их не сбрасывать
            self.w.grad.zero_()
            self.b.grad.zero_()
            self.loss_history.append(loss.item()) # достаём число из тензора

            if X_val is not None:
                with torch.no_grad():
                    self.val_loss_history.append(((X_val @ self.w + self.b - y_val) **
2).mean().item())

        return self # для того, чтобы можно было писать chain LinearRegression().fit(X, y)

    def predict(self, X):
        with torch.no_grad():
            return X @ self.w + self.b
```

Обучение: `lr=0.1`, `2000` эпох. MSE на train снизилась с `23616230` до `614068`, основное снижение приходится на первые сотни эпох. MSE на test минимальна около 450-й эпохи и дальше слегка растёт, это слабый признак переобучения.

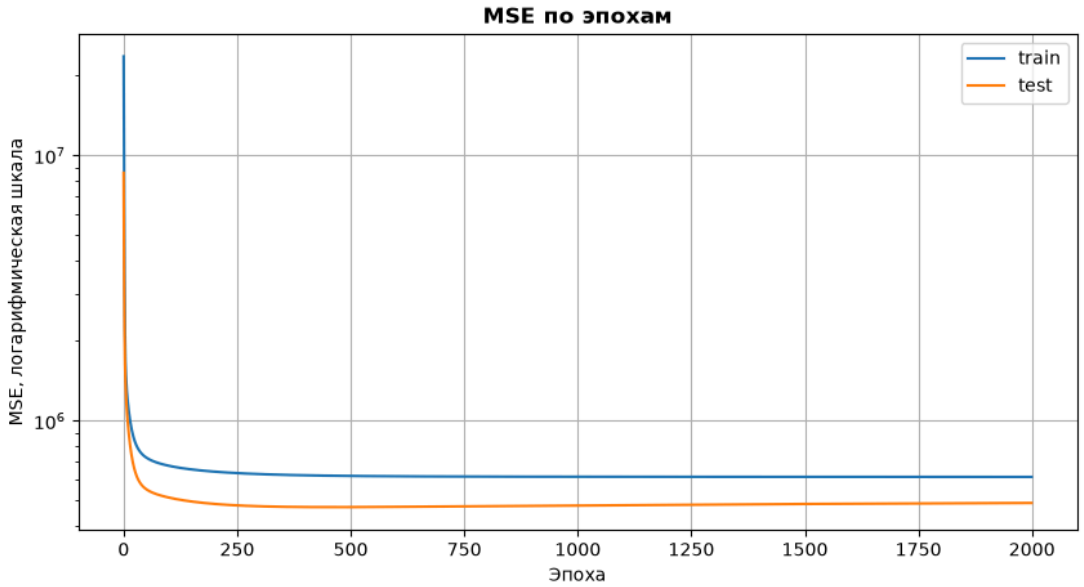


Рис. 2. MSE на train и test по эпохам, логарифмическая шкала

4.4. Результаты

Baseline предсказывает каждому дню среднее `cnt` по train. Среднее минимизирует MSE среди константных предсказаний.

	MAE	RMSE	R^2
LinearRegression, train	574.1	783.6	0.8435
Baseline, train	1610.1	1981.0	0.0000
LinearRegression, test	510.8	699.9	0.8515
Baseline, test	1522.3	1830.1	-0.0151

Модель снижает MAE на test в 3.0 раза, RMSE в 2.6 раза и объясняет 85% дисперсии cnt. R^2 baseline на test отрицательный, потому что среднее cnt в train ниже, чем в test. Метрики на test не хуже, чем на train, при этом в test всего 219 строк, и разница может быть случайной.

5. Часть B. Logistic Regression

5.1. Данные

Датасет содержит 8693 строки и 14 столбцов, дубликатов нет. Классы сбалансированы: True - 4378, False - 4315. Пропуски есть в 12 столбцах из 14, по 179-217 значений в каждом. Всего 2324 пропуска в 2087 строках, то есть в 24% строк. Удалять такие строки я не стал, потерялась бы четверть выборки.

Номер каюты из Cabin с целевой переменной линейно не связан: корреляция Пирсона -0.045, Спирмена -0.046. Доля Transported по диапазонам номера колеблется от 0.412 до 0.604 без монотонного тренда, такую зависимость логистическая регрессия не выучит.

5.2. Preprocessing

Признаки	Решение
PassengerId, Name	не используются
Cabin	разбит на палубу Deck и сторону Side, номер каюты не используется
HomePlanet, CryoSleep, Destination, VIP, Deck, Side	пропуск заменяется категорией Unknown, затем one-hot кодирование, первая категория отбрасывается
Age, RoomService, FoodCourt, ShoppingMall, Spa, VRDeck	пропуск заменяется медианой по train, затем стандартизация по train
Transported	переведён в 0.0 и 1.0

Категориальные пропуски заполняются до split, потому что Unknown - константа и от данных не зависит. Медиана считается по данным, поэтому количественные пропуски заполняются после split по train. Медиана train: Age - 27, траты - 0.

```
for c in CAT_COLS_B:
    X_b[c] = X_b[c].astype(object).fillna('Unknown').astype(str)
```

После кодирования получилось 26 признаков, train - 6086 строк, test - 2607. До заполнения в количественных признаках было 809 пропусков в train и 313 в test.

```
train_median_b = torch.nanmedian(X_train_b[:, num_idx_b], dim=0).values # Медиана каждого
столбца без учёта NaN. Возвращаем значение, а не только index

X_train_b_s = X_train_b.clone() # чтобы статистики каждый раз не обновлялись при повторном
запуске
X_test_b_s = X_test_b.clone()
X_train_b_s[:, num_idx_b] = torch.where(torch.isnan(X_train_b[:, num_idx_b]), train_median_b,
X_train_b[:, num_idx_b]) # медиана для train, если NaN, иначе исходное значение
X_test_b_s[:, num_idx_b] = torch.where(torch.isnan(X_test_b[:, num_idx_b]), train_median_b,
X_test_b[:, num_idx_b])


train_mean_b = X_train_b_s[:, num_idx_b].mean(dim=0)
train_std_b = X_train_b_s[:, num_idx_b].std(dim=0)
X_train_b_s[:, num_idx_b] = (X_train_b_s[:, num_idx_b] - train_mean_b) / train_std_b
X_test_b_s[:, num_idx_b] = (X_test_b_s[:, num_idx_b] - train_mean_b) / train_std_b
```

5.3. Модель

Модель считает вероятность класса 1 как $p = \sigma(Xw + b)$, где $\sigma(z) = \frac{1}{1+e^{-z}}$. Функция потерь - BCE:

$$L = -\frac{1}{n}\sum_i (y_i \ln p_i + (1 - y_i) \ln(1 - p_i))$$

`forward` считает вероятности с графом вычислений и используется в `fit`. `predict_proba` считает то же самое без графа, `predict` сравнивает вероятность с порогом `0.5`.

```
class LogisticRegression:
    def __init__(self, lr=0.1, n_epochs=1000, threshold=0.5):
        self.lr = lr
        self.n_epochs = n_epochs
        self.threshold = threshold
        self.w = None
        self.b = None
        self.loss_history = []
        self.val_loss_history = []

    @staticmethod
    def logLoss(p, y, eps=1e-7):
        p = p.clamp(eps, 1 - eps) # для того, чтобы при p = 0 не произошло log(+0) = -inf, то
есть диапазон p=[eps, 1 - eps]
        return -(y * torch.log(p) + (1 - y) * torch.log(1 - p)).mean()

    def fit(self, X, y, X_val=None, y_val=None):
        self.w = torch.zeros(X.shape[1], dtype=torch.float32, requires_grad=True)
        self.b = torch.zeros(1, dtype=torch.float32, requires_grad=True)
        self.loss_history = []
        self.val_loss_history = []
```

```
        for _ in range(self.n_epochs):
            p = self.forward(X) # перевод в вероятность от 0 до 1
            loss = self.logLoss(p, y)
            loss.backward() # считает градиенты в тензоре через PyTorch Autograd
            # через метод обратного распространения ошибки (backpropagation)

            with torch.no_grad(): # само обновление весов не должно попадать в граф вычислений,
иначе autograd начнёт дифференцировать шаг градиентного спуска.
                self.w -= self.lr * self.w.grad # обновление весов
                self.b -= self.lr * self.b.grad # обновление свободного коэффициента

            # backward() накапливает градиенты при каждом запуске, то есть аккумулирует
градиенты, если их не сбрасывать
            self.w.grad.zero_()
            self.b.grad.zero_()
            self.loss_history.append(loss.item()) # достаём число из тензора

            if X_val is not None:
                with torch.no_grad():
                    self.val_loss_history.append(self.logLoss(self.predict_proba(X_val),
y_val).item())
        return self



    def forward(self, X):
        return torch.sigmoid(X @ self.w + self.b)

    def predict_proba(self, X):
        with torch.no_grad():
            return self.forward(X)

    def predict(self, X):
        return (self.predict_proba(X) >= self.threshold).float()
```

Обучение: `lr=0.5`, `2000` эпох. BCE на train снизилась с `0.6931` до `0.4302`. Начальное значение равно $\ln 2$, потому что при нулевых весах модель всем даёт вероятность `0.5`. BCE на test идёт рядом с train и не растёт.

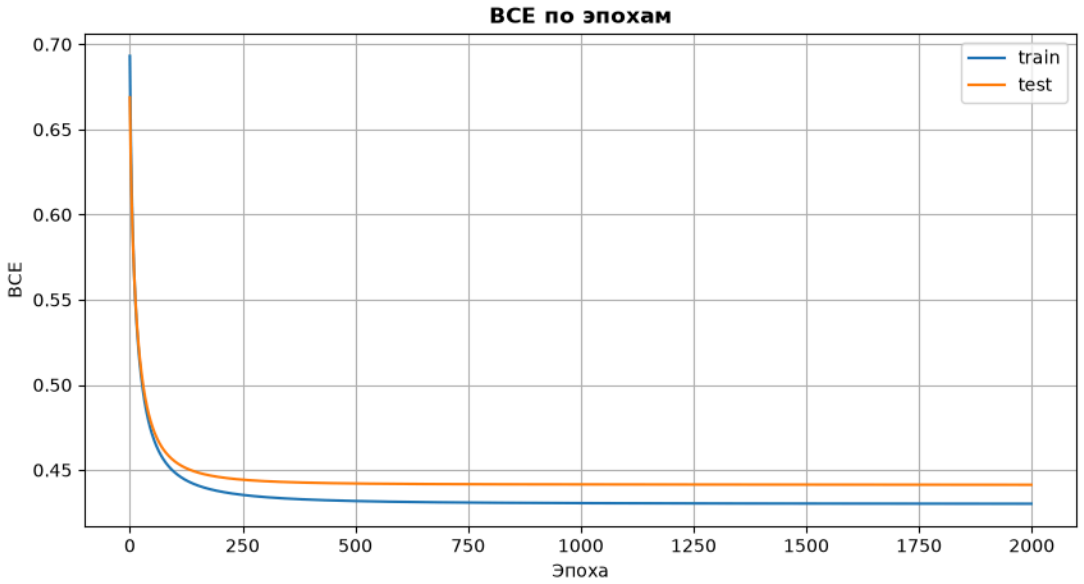


Рис. 3. BCE на train и test по эпохам

5.4. Результаты

Majority baseline предсказывает всем самый частый класс train, это класс `1` с долей `0.5089`. Для LogLoss baseline выдаёт всем вероятность `0.5089`.

	Accuracy	**Precision**	**Recall**	**F1**	**LogLoss**
LogisticRegression, train	`0.7920`	`0.7858`	`0.8127`	`0.7990`	`0.4302`
Majority, train	`0.5089`	`0.5089`	`1.0000`	`0.6745`	`0.6930`
LogisticRegression, test	`0.7848`	`0.7723`	`0.7970`	`0.7845`	`0.4413`
Majority, test	`0.4914`	`0.4914`	`1.0000`	`0.6590`	`0.6936`

Test	**Прогноз: 0**	**Прогноз: 1**
Факт: 0	TN = `1025`	FP = `301`
Факт: 1	FN = `260`	TP = `1021`

Модель верно классифицирует `78.5%` пассажиров test против `49.1%` у baseline. F1 baseline `0.659` получается только за счёт Recall `1.0`, потому что baseline всем предсказывает класс `1`. Ошибки распределены почти поровну: `301` ложноположительный ответ и `260` ложноотрицательных.

5.5. Анализ трат

От `61%` до `64%` пассажиров по каждой услуге ничего не тратили, у остальных траты доходят до `30000`. После `log1p` длинный хвост сжимается.

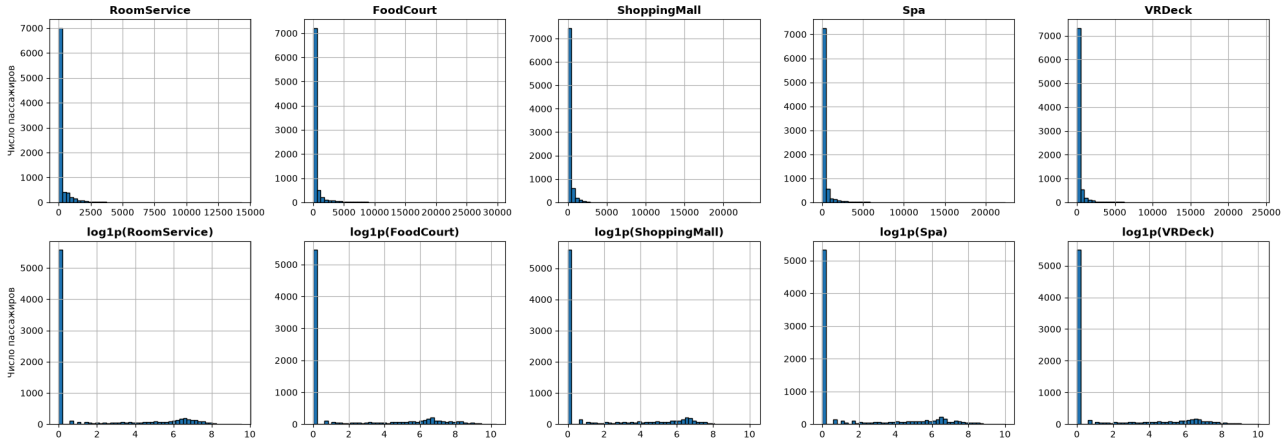


Рис. 4. Распределение трат до и после `log1p`

Тратившие пассажиры переносились в 2-2.5 раза реже: доля `Transported` `0.26-0.35` против `0.59-0.64`.

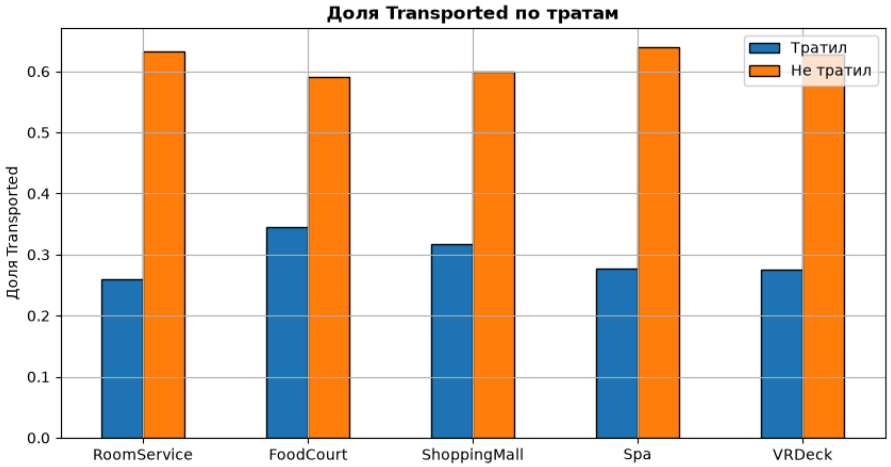


Рис. 5. Доля `Transported` у тративших и не тративших

6. Вывод

Я реализовал на PyTorch линейную и логистическую регрессию с обучением градиентным спуском, свой train/test split и preprocessing, статистики которого считаются только по train.

В части A я удалил `casual` и `registered`, потому что их сумма равна целевой переменной. Линейная регрессия на test дала MAE `510.8` и R^2 `0.8515` против MAE `1522.3` у baseline, который предсказывает среднее.

В части B я заполнил категориальные пропуски категорией `Unknown`, а количественные медианой по train. Логистическая регрессия на test дала Accuracy `0.7848` и F1 `0.7845` против `0.4914` и `0.6590` у majority baseline.

Что я бы улучшил:

- Применить `log1p` к тратам в части B, они сильно скошены.
- Добавить номер каюты, разбитый на диапазоны, и размер группы из `PassengerId`.

- В части A использовать хронологический split, потому что `cnt` растёт во времени и случайный split проверяет интерполяцию, а не прогноз.
- Подбирать `lr`, число эпох и порог на валидационной части train.